Dynamic Web Pages with Selenium

Week 8 · Documents module · Textbook Chapter 8

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 5, 7 & 9, 2026

This week at a glance

Most of the modern web renders its data with JavaScript *after* the HTML arrives. This week we drive a real browser to reach it.

Monday | Concepts & notebook intro

- Why JavaScript content is invisible to `requests`; Selenium as a browser you script

Wednesday | Notebook lab

- Work the Chapter 8 notebook: xkcd, a Wikipedia search, and an infinite-scroll page

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 8

Companion notebook

`ch-08-dynamic-pages.ipynb`

Bring a laptop

Wednesday is hands-on — a browser will open and click by itself.

1. Monday · Concepts

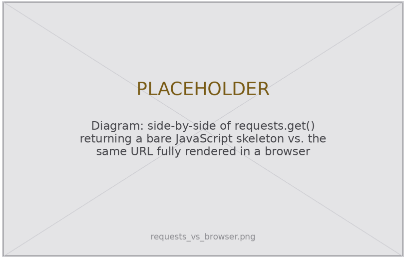
When static scraping fails

Last week's tools — `requests` + BeautifulSoup — work on **static** pages, whose content is fully present in the initial HTML.

But most modern sites are **dynamic**: their content is loaded or rendered by JavaScript *after* the HTML arrives. `requests.get()` hands you the skeleton *before* that code has run.

The data you see in your browser may simply not exist in the response. Diagnose it: compare `requests.get(url).text` to the page in your browser — if the content you want is missing, the page is dynamic.

The fix: a tool that *executes* JavaScript — a real browser, driven by code. That tool is **Selenium**.



Same URL, two very different payloads.

Textbook Ch. 8, "When Static Scraping Fails."

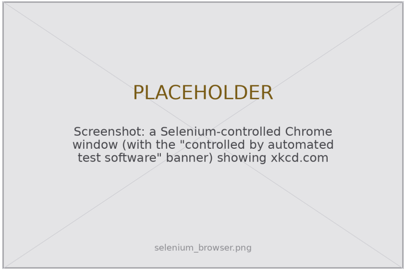
Selenium: a browser you drive with code

Selenium needs two things: the `selenium` library and a **browser driver** — a small program that lets Python control a specific browser.

Since v4.6 (late 2022), **Selenium Manager** finds or downloads the right driver automatically, so in practice you only `pip install selenium`. Creating a driver is a one-liner:

- `driver = webdriver.Chrome()` opens a real window
- every command you issue through `driver` happens *in that window*

You are not faking HTTP requests any more — you are **automating a browser**, the same one a person would use.



“Controlled by automated test software.”

Textbook Ch. 8, “Setting Up Selenium.”

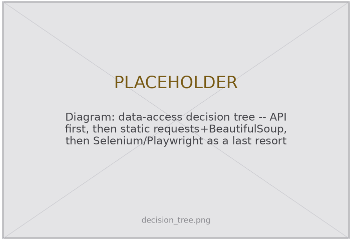
A powerful last resort — and not a loophole

Selenium is slow, resource-intensive, and brittle. Use it only when the simpler tools cannot. The decision tree:

1. **API?** Use it — fastest, most structured.
2. **In the static HTML?** `requests` + BeautifulSoup.
3. **Rendered by JavaScript?** *Now* reach for a browser.

Playwright (Microsoft) is the modern alternative — auto-waiting, faster, async — but Selenium is more widespread and fine for coursework.

Researchers increasingly *need* automation as platforms enclose once-open data (Freelon 2018) — which raises the stakes, not lowers them.



Automation is not a loophole

A `robots.txt` rule and a site's ToS still apply when a real browser does the clicking. (Ch. 2)

Textbook Ch. 8, "When to Use Selenium vs. APIs" & "The Fragility Problem."

What the notebook covers

This week's companion notebook builds the workflow one interaction at a time:

- **Launch & locate** — start a driver, find elements by ID, name, CSS selector, and XPath (on xkcd)
- **Extract** — read hidden attributes like an image's `alt` and `title` (the xkcd hover-text joke)
- **Interact** — click a button, type a Wikipedia search, and submit it like a user
- **Wait & scroll** — replace blind `time.sleep()` with `WebDriverWait`; harvest an infinite-scroll page
- **Hand off** — pass the rendered `page_source` to BeautifulSoup, then `driver.quit()`

Connecting to Ch. 2 (Ethics)

Everything from the ethics chapter applies with full force here. Before you automate *any* site, check its `robots.txt` and Terms of Service — especially anything logged-in or ToS-restricted, where the temptation is greatest.

2. Wednesday · Notebook Lab

Launch a driver, then locate elements

Four strategies find elements; `find_element` returns the first match, `find_elements` returns a list:

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome() # Selenium Manager fetches the driver
driver.get("https://xkcd.com")

comic = driver.find_element(By.ID, "comic") # by id
nav = driver.find_elements(By.CSS_SELECTOR, "ul.comicNav li a") # -> a list
title = driver.find_element(By.XPATH, "//div[@id='ctitle']") # by XPath
print(title.text) # the title of the current comic
```

Prefer a stable anchor — an `id` or a semantic `class` — over a brittle, position-based path.

Textbook Ch. 8, "Navigating and Finding Elements."

Read attributes, then act like a user

```python
# xkcd hides a joke in the image's title (hover) attribute
img = driver.find_element(By.XPATH, "//div[@id='comic']//img")
print(img.get_attribute("alt")) # the accessible description
print(img.get_attribute("title")) # the mouseover punchline

# Anything a user can do, Selenium can do -- e.g. click "Random":
btn = driver.find_element(
    By.XPATH, "//ul[@class='comicNav']//a[contains(text(),'Random')]")
btn.click()
```

`get_attribute()` reads *any* HTML attribute (even ones you never see); `.text` reads the visible text.

Textbook Ch. 8, "Extracting Data" & "Simulating Interactions."

Type, submit — then wait *well*

Fill a search box, submit, and wait for the exact condition instead of guessing a duration:

```
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver.get("https://www.wikipedia.org")
box = driver.find_element(By.NAME, "search")
box.send_keys("Colorado Buffaloes") # type like a user
box.send_keys(Keys.RETURN) # submit the form

WebDriverWait(driver, 10).until( # not a blind time.sleep(2)!
    EC.presence_of_element_located((By.ID, "firstHeading")))
print(driver.title) # -> Colorado Buffaloes - Wikipedia
```

`WebDriverWait` polls every 0.5 s, returns *the instant* the condition holds, and raises `TimeoutException` if it never does — faster, and it fails loudly. Common conditions: `presence_of_element_located`, `visibility_of_element_located`, `element_to_be_clickable`.

Textbook Ch. 8, "Typing and Searching" & "Waiting for Dynamic Content."

Scraping an infinite-scroll page

Scroll to the bottom, wait, and stop when the page stops growing:

```python
def scrape_infinite_scroll(driver, max_scrolls=20, pause=2):
    last = driver.execute_script("return document.body.scrollHeight")
    for i in range(max_scrolls):
        driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
        time.sleep(pause) # let new content load
        new = driver.execute_script("return document.body.scrollHeight")
        if new == last: # height stopped growing -> the end
            break
        last = new
    return driver.page_source # the fully-loaded HTML
```

Set a sane `max_scrolls`. **Proportionality** (Ch. 2): collect only what your question needs — not an entire 10,000-post feed.

Textbook Ch. 8, "Scraping Infinite Scroll Pages."

Selenium renders; BeautifulSoup reads

Get the page into the state you need, hand the rendered HTML to BeautifulSoup, and **always** clean up:

```
from bs4 import BeautifulSoup

try:
    driver.get("https://example-search-site.com")
    # ...scroll / click / search until the page is ready...
    soup = BeautifulSoup(driver.page_source, "html.parser") # rendered HTML
    for item in soup.find_all("div", class_="result-item"):
        title = item.find("h3")
        if title is None: # same defensive habit as static scraping
            continue # skip a malformed item instead of crashing
        print(title.get_text(strip=True))
finally:
    driver.quit() # each driver is a whole browser process
```

Selenium is the **hands** that navigate; BeautifulSoup is the **eyes** that read. `try/finally` guarantees the browser closes even when parsing throws.

Textbook Ch. 8, "Passing to BeautifulSoup" & "A Practical Selenium Workflow."

Lab: work these, then show-and-tell

Work through the chapter's exercises in pairs:

1. Extract JavaScript-loaded data; compare it to what `requests.get()` returns
2. Automate a multi-step search: navigate, query, submit, extract
3. Categorize three sites as static, partially, or fully dynamic
4. "Load More" button: click in a loop until it disappears — how many items were hidden?
5. Time `requests` vs. headed vs. headless Selenium; when is the tradeoff worth it?
6. 5617: instrument a JS-heavy research site with both approaches; a ~500-word memo engaging Mitchell (2018)

Show-and-Tell

Come ready to share:

- a challenging bug
- interesting data you pulled
- a provocation for a research design

The Twitter scraper that worked in 2019 was dead by 2024.

Browser automation is borrowed time — write it defensively, and expect it to break.

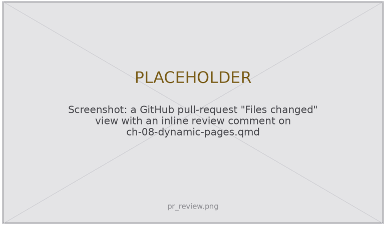
3. Friday · Textbook Revisions

Improving Chapter 8 together

Today we make the book better. Each of you opens a **pull request** against `ch-08-dynamic-pages.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



Contributions & reviews count toward the Textbook Revisions grade (30%).

A revision menu for Chapter 8

High-value targets — pick one and scope it to a single PR:

- **Test the code against live sites.** The xkcd and Wikipedia examples drift. Run the notebook headless, report what broke (a renamed class, a new consent banner), and fix the selector — the chapter's own "Fragility Problem."
- **Close a gap in "Common Issues to Debug."** Add a worked `StaleElementReferenceException` reproduction, or a headed-vs-headless content difference caused by `navigator.webdriver` detection.
- **Write the Playwright counterpart.** The chapter *names* Playwright as the modern alternative but shows no code — add a short side-by-side of the search example.
- **Add a figure.** A clean API → static → Selenium decision-tree diagram, or an annotated shot of a driver-controlled browser.
- **Strengthen an exercise.** Add expected output, a rubric, or a starter cell (e.g. a public infinite-scroll demo page) so Exercise 1 or 4 is self-checking.
- **Extend "Further Reading."** A public-interest case study that turned to browser automation after an API closed — tying the "Public Interest" callout to the post-API enclosure.

Targets drawn from the chapter's callouts, "Common Issues," and "Further Reading."

What makes a good revision & a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Extracting data from PDFs** — when the data isn't on a web page at all.

Key takeaways

1. Static tools see the HTML **skeleton**; dynamic pages render their data with JavaScript *after* it arrives.
2. Selenium drives a **real browser** — locate elements (ID / name / CSS / XPath), then click, type, and scroll like a user.
3. Wait on **conditions** with `WebDriverWait`, not a blind `time.sleep()` — it's faster and fails loudly.
4. Render with Selenium, extract with BeautifulSoup — hands and eyes — and `driver.quit()` every time.
5. Selenium is a slow, fragile **last resort**: API → static → browser. Automation never suspends `robots.txt` or a site's ToS.

Reading: Textbook Ch. 8. Related: Mitchell (2018); García et al. (2020); Perriam et al. (2020).